Putting a Journey on the APRS Map

A Python tracker that gives virtual journeys a callsign, a timetable, and a little unpredictability- by ***Erwin, PA3EFR***

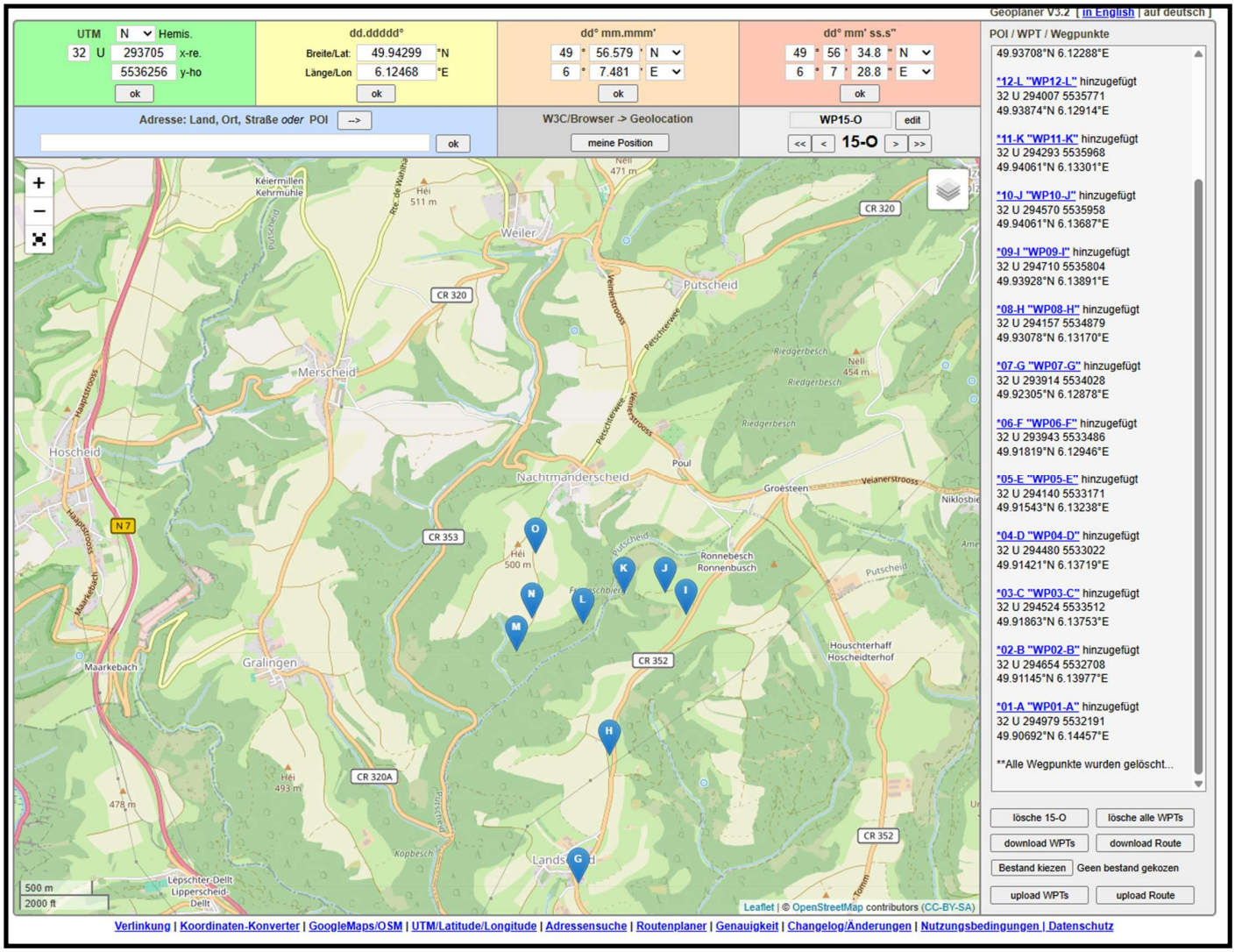
APRS maps are great for seeing where stations are—but what if you want to demonstrate a journey, follow several moving objects at once, or have a station appear to travel along a planned route? This Python script takes a waypoint list and turns it into a sequence of APRS position reports sent to APRS-IS. It is a route simulator, rather than a GPS receiver: the positions come from a file, not from a live device in a car, bicycle, or rucksack.

The first use of the script was to deceive APRS-followers during a Mystery Survival Expedition and it worked like a champ.

One script, several journeys

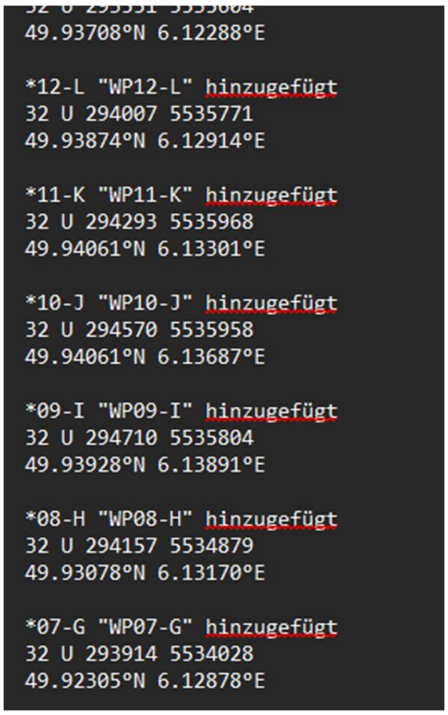
At startup, the program asks how many objects you want to track. For each one, you enter a callsign, choose a transport type—car, bicycle, or walking—set the reporting interval, optionally choose when the first position should be uploaded, and add a short comment. Each object gets its own waypoint file, named from the callsign without any SSID: for example, a callsign such as PA3XYZ-7 uses “Waypoints_pa3xyz.txt”.

To make this waypoint file the user could go to https://geoplaner.de and create a route by clicking on the map. On the right the list is created with the last click on top of the list. The script will take care of the correct direction of the route.



Copy the right hand frame content to a text editor and create the Waypoints_<callsign>.txt file. Save this file in the same directory as where the Python script is running.

The objects run concurrently in separate threads. So a car, a cyclist, and a walker can all be moving across aprs.fi at roughly the same time, each with its own route and reporting interval. The console output is protected by a lock, helping keep messages from the different objects readable instead of turning into one long, scrambled pile of coordinates.



Waypoints in, positions out

The waypoint file is read line by line. Any line containing a coordinate pair in the expected degrees format is treated as a waypoint; other lines, including the UTM line described in the file format, are ignored. At least two waypoints are required. The script reverses the collected list because its expected source files have the final waypoint at the top, so check the file order before setting off.

Between each pair of waypoints, the script calculates the distance using a great-circle distance formula. It then creates intermediate positions along the straight line between the coordinates. The intermediate spacing is deliberately **not** perfectly uniform: random weights vary the steps a little, so the trail is less mechanically regular. The endpoints remain fixed, but the route between them is a simplified straight-line path—not road-aware navigation.

A car, a bike, or a pair of walking shoes

The selected transport type determines both the APRS map symbol and the speed range used for the simulation. The script uses 50–120 km/h for a car, 10–25 km/h for a bicycle, and 1–5 km/h for walking. For each route segment it chooses a random speed within the relevant range, estimates how long that segment should take, and works out how many position points to generate from the chosen interval.

There is a little variation in the timing too: waits between reports are randomly adjusted to between 80% and 120% of the requested interval. This makes the stream less metronomic, while still following the general pace chosen for the object. The total journey time and approximate finish time are displayed before tracking begins. Treat those times as estimates, not promises—the actual segment speeds are random, so the simulated journey may finish earlier or later.

Choose when the journey appears

The first upload can be sent immediately or scheduled for a local time in HH:MM format. If that time has already passed today, the script interprets it as tomorrow. Only the first upload is delayed; once the journey starts, the remaining positions follow the normal interval. This is a handy way to prepare a demonstration in advance without having to be at the keyboard at precisely the right moment.

APRS-IS, passcodes, and the console report

The script builds uncompressed APRS position packets and submits them to APRS-IS using the Python library aprslib over TCP port 14580. It does not control a radio transmitter or obtain live GPS fixes. Passcodes are read from “APRS_Codes.txt”, with callsigns matched without their SSID; if a callsign is not found, the program asks the operator to enter the passcode.

As it runs, the console reports segment distances, selected speeds, estimated segment durations, generated coordinates, and whether each upload succeeded. At the end, it prints a success count for each callsign and an aprs.fi link. That makes it fairly easy to see which simulated journey finished cleanly and which one may need a closer look.

A few sensible operating habits

Before running the script, make sure the waypoint files match the callsigns and contain at least two valid coordinate lines. Check that the waypoint order is what you intend, that the chosen reporting interval is appropriate, and that you have the required APRS-IS passcode. Because the script publishes positions under the configured callsigns, use it responsibly and avoid creating misleading impressions about the live location of a real station or person.

For radio amateurs, the fun here is in experimenting with APRS presentation and timing: multiple objects, different apparent speeds, and staggered starts—all from a waypoint file and a Python terminal. Just remember that the dots on the map represent a programmed simulation, not necessarily a real journey.

73—and may your virtual convoy arrive on time, give or take a random interval.

Repository: https://github.com/PA3EFR/aprs_tracking_multi_objects/tree/main/MultiObject